ArtiShop · TP2 Git

Compte rendu des workflows

Zouhair Fouiguira Yassine Belhamadia Zainab Achak Wiam Bzioui

Fork de l'équipe · ZOUHAIRFGRA/fsr_ArtiShop

Pièces jointes · Le graphe Git complet accompagne le rapport dans `tp2-graph.txt`. Capture de `main` à `c43230c` avant le commit de publication. Le journal d'incidents est fourni séparément du dépôt.

A. Centralized Workflow

Observation. Au commit `8e1afb9`, quatre entrées partageaient l'ID 7. Une recherche par ID utilisant `find` renvoie la première entrée correspondante, donc l'API n'expose pas les autres à cette URL. `4361716` a rendu les IDs uniques. La reprise a remplacé les entrées de démonstration par les quatre produits demandés avec les IDs 7 à 10 et conservé les six produits d'origine.

QA.1 Les nombres de tentatives par personne ne sont pas établis par le reflog disponible. Dans le scénario de quatre pushs partant du même commit, un premier push peut passer et les trois autres doivent intégrer la nouvelle pointe avant de repousser. Si un collègue pousse pendant cette intégration, une autre tentative peut être refusée. Avec quinze développeurs, le risque de retries et le coût de coordination augmentent.

QA.2 Git compare et fusionne le texte. Deux objets différents peuvent avoir le même ID et former un fichier JavaScript valide. Un test d'unicité sur le tableau, exécuté dans la CI avant intégration, aurait détecté le doublon. Une revue du diff offre un contrôle complémentaire.

QA.3 Un commit incomplet peut rendre `main` instable et une résolution manuelle précipitée peut supprimer une modification d'un collègue.

B. Feature Branch et pull requests

Les PR #1 Catégories et #2 Recherche sont parties de la même base. Le tri alphabétique français et la recherche dans la catégorie ont été ajoutés après les commentaires. Les catégories ont été fusionnées en premier. La reprise de la recherche sur `origin/main` a provoqué un conflit dans `src/app.js`. Les deux routes ont été conservées et vérifiées avant la seconde fusion.

QB.1 Une PR isole le travail de `main` et donne un emplacement commun pour lire le diff, discuter, tester et vérifier les résultats de CI avant fusion. Les corrections poussées sur la branche mettent la même PR à jour.

QB.2 Une approbation concerne le contenu relu à un moment donné. La base a changé quand la première PR a été fusionnée. Le membre 2, auteur de la branche recherche, réconcilie les deux routes car il connaît l'intention du changement et peut valider la version intégrée. Le membre 1 vérifie ensuite le résultat avant fusion.

QB.3 Le rebase réécrit les commits et la nouvelle pointe n'est plus descendante de l'ancienne pointe distante. Le push simple est refusé en non-fast-forward. `--force-with-lease` limite l'écrasement à une référence distante qui correspond toujours à la valeur attendue. `--force` n'effectue pas cette vérification.

QB.4 Le relecteur vérifie la conformité au besoin, le diff et les effets de bord, puis lance les tests et essaie les URLs avec des cas normaux, vides et sans résultat. Il vérifie que les retours de revue sont traités et recontrôle le changement après un conflit. Ici, les contrôles ont couvert le tri français, le nom, la catégorie, la casse et l'absence de résultat.

C. Gitflow

Le correctif urgent est parti de `main`, a été intégré dans `main` et `develop` et porte le tag `v1.0.1`. Deux intégrations concurrentes ont ensuite été reproduites depuis la même pointe de `develop`. Le second push a été refusé. Le pull sans rebase a produit un conflit dans `promos.js`, résolu en conservant `BLACKFRIDAY30` et `EID15`.

La branche `release/1.1` a été intégrée dans les deux branches permanentes après validation technique. Le tag annoté est `tp2/v1.1.0` car `v1.1.0` désignait déjà le livrable du TP1. La version applicative est bien `1.1.0`.

QC.1 Le bug concerne la version livrée, donc le correctif part de `main` pour éviter d'embarquer les fonctionnalités inachevées de `develop`. Le membre 3 peut créer sa branche sans attendre la création de `develop`.

QC.2 Réintégrer le correctif dans `develop` garantit que la branche de préparation de la prochaine version en hérite. Sinon, `release/1.1` pourrait encore accepter le même code deux fois et le défaut pourrait réapparaître lors d'une intégration ultérieure.

QC.3 Il est possible de signaler à l'avance l'édition de la même liste, de synchroniser plus tôt, de diviser les ajouts en fichiers distincts ou de désigner un intégrateur pour le registre des promos. Un test sur la configuration finale complète la résolution textuelle.

Vérification de la release. Les six tests passent. Pour un tapis à 1 800 MAD, la première application de RAMADAN20 donne 1 440 MAD et la seconde reçoit HTTP 400. BLACKFRIDAY30 donne 1 260 MAD et EID15 donne 1 530 MAD, chacun sur un panier neuf. Avant le correctif, la double application de RAMADAN20 donnait 1 152 MAD.

D. Bilan de l'équipe

Critère	Centralized	Feature Branch	Gitflow
Branches permanentes	`main`	`main`	`main`, `develop`
Relecture	Aucune imposée	PR avant fusion	Selon les règles d'intégration et de release
Refus et conflits	Comptes individuels inconnus, doublon d'ID confirmé	1 refus, 1 conflit de rebase	1 refus, 1 conflit dans les promos
Résolution	Auteur du push refusé	Auteur de la PR en conflit	Intégrateur sur `develop`
Attente	Intégration des pushs concurrents	Revue et résolution de la seconde PR	Correctif et fonctionnalités avant release
Complexité	Faible	Intermédiaire	Plus élevée
Adapté à	Petit travail peu concurrent	Équipe qui livre souvent	Versions planifiées et correctifs séparés

Les durées d'attente individuelles de la séance ne sont pas mesurées. Les incidents B et C correspondent à la reprise contrôlée et figurent dans le journal.

Pour ArtiShop, nous recommandons le **Feature Branch Workflow**.
Les quatre membres peuvent développer en parallèle avec des PR courtes.
Une revue et un test d'unicité des IDs peuvent éviter le problème de la partie A.
Le conflit de la partie B reste localisé à la branche de recherche.
Des tests de CI avant fusion protègent la version unique en ligne.
Une branche courte depuis `main` suffit pour un correctif urgent sans maintenir un `develop` permanent.